

Laporan Praktikum Kontrol Cerdas

(Praktikum 15 SKICIT-LEARNND)

Nama : Bujang Setia Aditama
NIM 234308066
Kelas : TKA – 6C
Github : <https://github.com/BujangSA>

1. Judul : SKICIT-LEARN

2. Pendahuluan

Perkembangan teknologi Artificial Intelligence (AI) dan Machine Learning (ML) telah memberikan dampak signifikan dalam berbagai bidang, termasuk pengolahan data, sistem prediksi, serta computer vision. Salah satu library yang banyak digunakan dalam implementasi machine learning berbasis Python adalah Scikit-learn. Library ini menyediakan berbagai algoritma pembelajaran mesin seperti klasifikasi, regresi, clustering, serta evaluasi model yang mudah digunakan dan efisien.

Scikit-learn dibangun di atas library numerik Python seperti NumPy dan SciPy, sehingga mampu melakukan pemrosesan data secara cepat dan akurat. Dalam konteks pengolahan citra dan gesture recognition, Scikit-learn sering digunakan untuk membangun model klasifikasi berbasis fitur yang telah diekstraksi sebelumnya, misalnya dari hasil deteksi landmark tangan menggunakan MediaPipe.

Pada praktikum ini, SKICIT-LEARNN difokuskan pada implementasi algoritma machine learning untuk mendeteksi dan mengklasifikasikan gestur tangan Sistem Isyarat Bahasa Indonesia (SIBI), serta mendukung analisis posisi tubuh seperti berdiri dan duduk. Dengan menggabungkan teknik computer vision dan machine learning, sistem mampu melakukan klasifikasi secara real-time menggunakan webcam sebagai input.

Melalui kegiatan ini, mahasiswa diharapkan mampu memahami alur kerja machine learning mulai dari ekstraksi fitur, pelatihan model, pengujian akurasi, hingga implementasi sistem klasifikasi berbasis kamera secara langsung

.

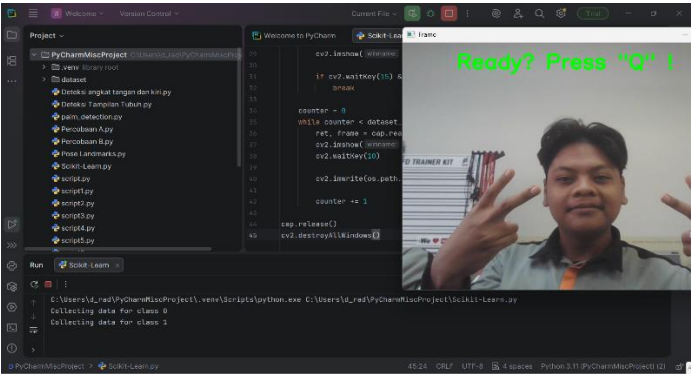
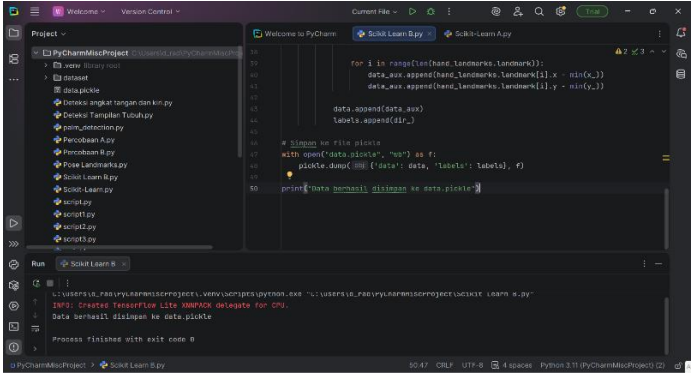
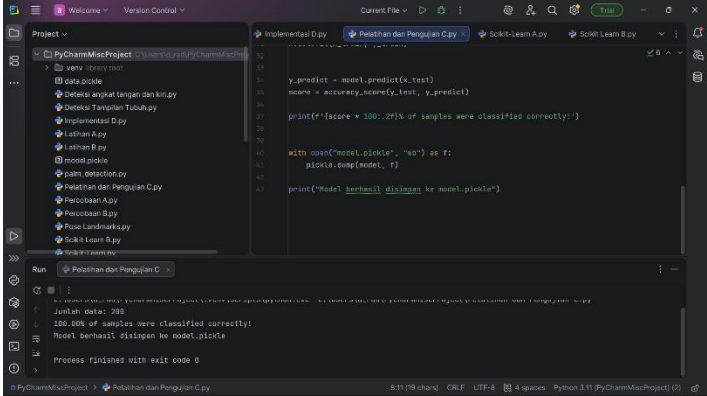
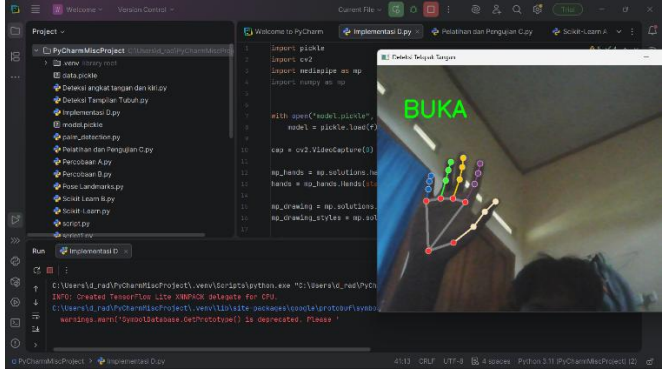
3. Tujuan

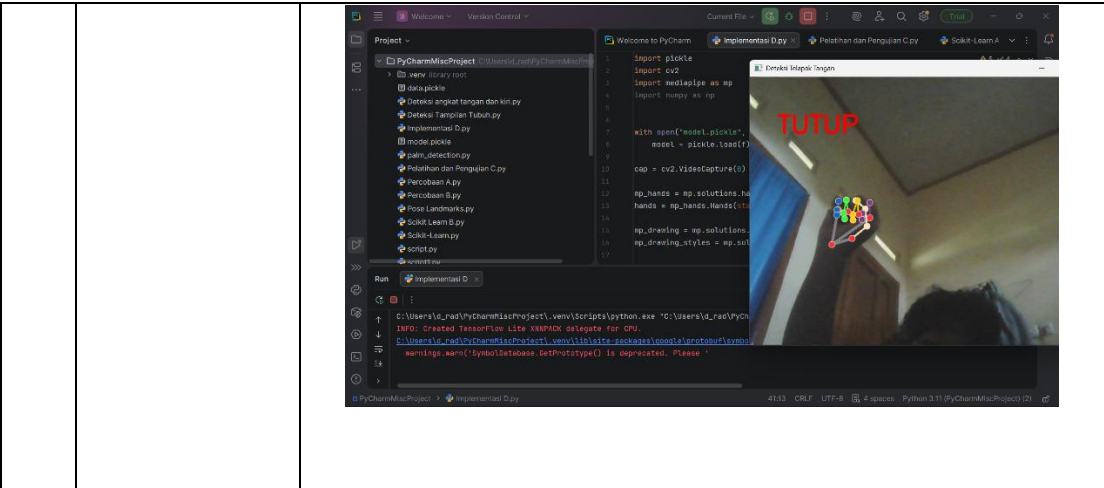
- 3.1 Memahami konsep dasar computer vision menggunakan Python.
- 3.2 Mempelajari penggunaan library Scikit-learn dalam proses klasifikasi data. Menggunakan MediaPipe Pose untuk mendeteksi pose tubuh manusia secara 2D.
- 3.3 Mengolah data hasil ekstraksi fitur menjadi dataset yang siap dilatih.
- 3.4 Mengintegrasikan model machine learning dengan sistem deteksi real-time berbasis webcam.
- 3.5 Membangun sistem pengenalan huruf alfabet SIBI secara real-time.

4. Manfaat

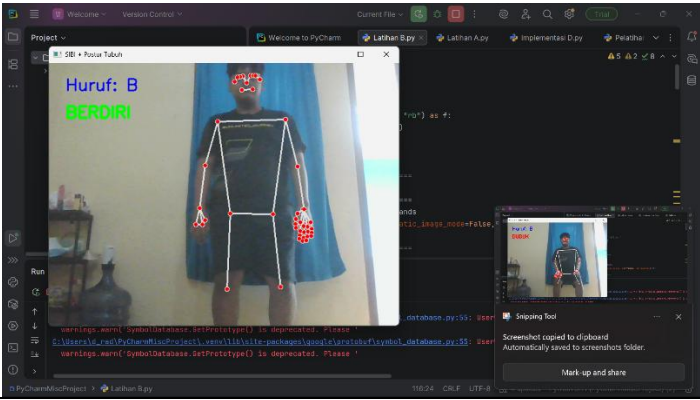
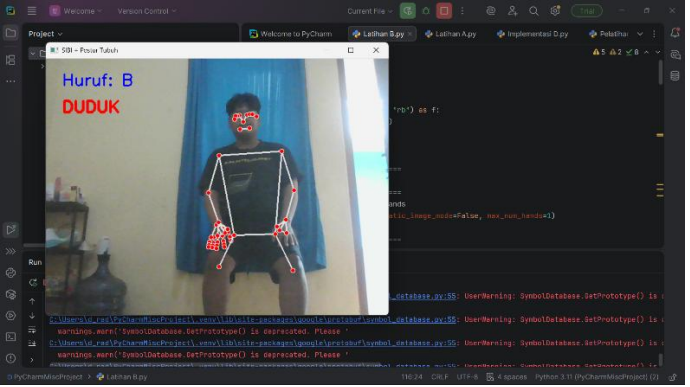
- 4.1 Meningkatkan pemahaman tentang pemrosesan citra digital dan computer vision.
- 4.2 Meningkatkan pemahaman SKICIT-LEARN
- 4.3 Melatih kemampuan pemrograman Python dalam pengolahan data visual.
- 4.4 Memberikan dasar pengembangan sistem berbasis gesture recognition
- .

5. Output hasil running program

No.	Tugas	Hasil dan Dokumentasi
1	Pengumpulan data menggunakan webcam	
2	Ekstraksi dari dataset	
3.	Pelatihan dan Pengujian	
4.	Implementasi	



5. Latihan B mendeteksi seseorang apakah berdiri atau duduk



6. Analisis program

6.1 Pengumpulan data menggunakan webcam

```
1 import os
2 import cv2
3
4 DATA_DIR = r"C:\Users\d_nad\PyCharmMiscProject\.venv\Lib\site-packages\sklearn\datasets\tests\dataset"
5
6 os.makedirs(DATA_DIR, exist_ok=True)
7
8 number_of_classes = 2
9 dataset_size = 100
10
11 cap = cv2.VideoCapture(0)
12
13 for j in range(number_of_classes):
14
15     class_path = os.path.join(DATA_DIR, str(j))
16     os.makedirs(class_path, exist_ok=True)
17
18     print(f'Collecting data for class {j}')
19
20     while True:
21         ret, frame = cap.read()
22         if not ret:
23             print("Kamera tidak terbaca")
24             break
25
26         cv2.putText(frame, 'Ready? Press "Q" !',
```

```
28             cv2.FONT_HERSHEY_SIMPLEX,
29             1.3,
30             (0, 255, 0),
31             3)
32
33         cv2.imshow('Frame', frame)
34
35         if cv2.waitKey(15) & 0xFF == ord('q'):
36             break
37
38         counter = 0
39         while counter < dataset_size:
40             ret, frame = cap.read()
41             if not ret:
42                 break
43
44             cv2.imshow('Frame', frame)
45             cv2.waitKey(10)
46
47             cv2.imwrite(os.path.join(class_path, f'{counter}.jpg'), frame)
48
49             counter += 1
50
51 cap.release()
52 cv2.destroyAllWindows()
```

Program yang dikembangkan bertujuan untuk melakukan proses pengumpulan dataset citra secara otomatis menggunakan kamera (webcam) sebagai sumber input. Dataset yang diperoleh selanjutnya digunakan sebagai data latih dalam proses klasifikasi berbasis machine learning menggunakan Scikit-Learn. Pada tahap awal, program mengimpor library os dan cv2. Library os berfungsi untuk pengelolaan direktori dan sistem berkas, sedangkan cv2 (OpenCV) digunakan untuk mengakses perangkat kamera serta melakukan pengolahan citra digital secara real-time.

Variabel DATA_DIR didefinisikan sebagai lokasi penyimpanan dataset. Program kemudian menggunakan fungsi os.makedirs() dengan parameter exist_ok=True untuk memastikan direktori utama tersedia tanpa menimbulkan error apabila folder telah dibuat sebelumnya. Selanjutnya, ditentukan jumlah kelas (number_of_classes = 2) dan jumlah data per kelas (dataset_size = 100). Hal ini menunjukkan bahwa sistem akan membuat dua kategori data dengan masing-masing berisi seratus citra. Struktur direktori yang dihasilkan

secara otomatis akan berbentuk folder terpisah untuk setiap kelas, sehingga memudahkan proses pelabelan dan pengolahan data pada tahap pelatihan model.

Proses pengambilan data dilakukan dengan mengaktifkan webcam melalui perintah `cv2.VideoCapture(0)`, di mana indeks 0 menunjukkan penggunaan kamera default pada perangkat. Program kemudian melakukan perulangan berdasarkan jumlah kelas yang telah ditentukan. Untuk setiap kelas, dibuat sub-direktori khusus sebagai tempat penyimpanan citra sesuai label kelas tersebut. Pendekatan ini memungkinkan proses pengelompokan data secara sistematis dan terstruktur.

Pada bagian utama program, digunakan perulangan `while True` untuk membaca frame video secara kontinu dari kamera. Setiap frame yang berhasil ditangkap akan disimpan dalam variabel `frame`. Apabila pembacaan frame gagal, sistem akan menampilkan pesan kesalahan dan menghentikan proses untuk mencegah terjadinya kegagalan berkelanjutan. Selain itu, program menampilkan teks instruksi pada layar menggunakan fungsi `cv2.putText()`, yang berfungsi sebagai panduan bagi pengguna sebelum proses pengambilan data dimulai.

Secara keseluruhan, program ini berperan sebagai modul awal dalam sistem machine learning, yaitu pada tahap akuisisi data (data acquisition). Kualitas dan jumlah data yang dikumpulkan pada tahap ini sangat berpengaruh terhadap performa model klasifikasi yang akan dibangun. Dengan struktur direktori yang terorganisasi dan proses pengambilan citra secara real-time, program ini telah memenuhi prinsip dasar pengumpulan dataset terlabel untuk kebutuhan pelatihan model berbasis Scikit-Learn.

6.2 Ekstraksi data dari dataset

```
1 import os
2 import pickle
3 import mediapipe as mp
4 import cv2
5
6 mp_hands = mp.solutions.hands
7 hands = mp_hands.Hands(static_image_mode=True, max_num_hands=1)
8
9 DATA_DIR = r"C:\Users\d_rad\PyCharmMiscProject\.venv\Lib\site-packages\sklearn\datasets\tests\dataset"
10
11 data = []
12 labels = []
13
14 for dir_ in os.listdir(DATA_DIR):
15     for img_path in os.listdir(os.path.join(DATA_DIR, dir_)):
16
17         data_aux = []
18         x_ = []
19         y_ = []
20
21         img = cv2.imread(os.path.join(DATA_DIR, dir_, img_path))
22
23         if img is None:
24             continue
25
26         img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
28 Welcome to PyCharm results.multi_hand_landmarks:
29     for hand_landmarks in results.multi_hand_landmarks:
30
31         for i in range(len(hand_landmarks.landmark)):
32             x = hand_landmarks.landmark[i].x
33             y = hand_landmarks.landmark[i].y
34
35             x_.append(x)
36             y_.append(y)
37
38         for i in range(len(hand_landmarks.landmark)):
39             data_aux.append(hand_landmarks.landmark[i].x - min(x_))
40             data_aux.append(hand_landmarks.landmark[i].y - min(y_))
41
42         data.append(data_aux)
43         labels.append(dir_)
44
45
46
47 with open("data.pickle", "wb") as f:
48     pickle.dump(obj={'data': data, 'labels': labels}, f)
49
50 print("Data berhasil disimpan ke data.pickle")
```

Program yang dikembangkan merupakan tahap lanjutan setelah proses pengumpulan dataset, yaitu tahap ekstraksi fitur dari citra tangan menggunakan library MediaPipe. Pada bagian awal, program mengimpor beberapa library utama, yaitu os untuk pengelolaan direktori, pickle untuk penyimpanan data hasil ekstraksi, mediapipe sebagai modul deteksi landmark tangan, serta cv2 (OpenCV) untuk pengolahan citra digital. Inisialisasi dilakukan dengan memanggil modul mp.solutions.hands, kemudian membuat objek Hands dengan parameter `static_image_mode=True` dan `max_num_hands=1`, yang menunjukkan bahwa sistem memproses gambar statis dan hanya mendeteksi satu tangan pada setiap citra.

Variabel `DATA_DIR` digunakan untuk menunjuk lokasi dataset yang sebelumnya telah dikumpulkan. Program kemudian menyiapkan dua buah list, yaitu data untuk menyimpan fitur numerik hasil ekstraksi landmark dan labels untuk menyimpan label kelas dari setiap gambar. Proses utama dilakukan melalui perulangan direktori menggunakan `os.listdir(DATA_DIR)`, yang memungkinkan sistem membaca setiap folder kelas (misalnya kelas 0 dan 1). Untuk setiap folder, program kembali melakukan iterasi terhadap seluruh file gambar yang ada di dalamnya.

Setiap citra dibaca menggunakan fungsi `cv2.imread()`, kemudian dilakukan pengecekan apakah citra berhasil dimuat. Jika gambar tidak terbaca, program akan melewati file tersebut untuk mencegah terjadinya error selama proses komputasi. Selanjutnya, citra dikonversi dari format BGR ke RGB menggunakan `cv2.cvtColor()` karena MediaPipe bekerja dengan format warna RGB. Proses ini penting untuk memastikan kompatibilitas data citra dengan model deteksi landmark tangan.

Setelah konversi warna dilakukan, citra diproses oleh model MediaPipe untuk mendeteksi titik-titik landmark tangan. Landmark tersebut terdiri dari koordinat posisi (x dan y) yang merepresentasikan struktur geometris tangan. Koordinat tersebut kemudian dikumpulkan dan dinormalisasi untuk mengurangi pengaruh pergeseran posisi tangan dalam frame. Hasil normalisasi disimpan dalam variabel `data_aux`, yang selanjutnya dimasukkan ke dalam list data, sedangkan nama folder kelas dimasukkan ke dalam list labels.

Secara keseluruhan, program ini berfungsi sebagai modul ekstraksi fitur dalam sistem klasifikasi berbasis machine learning. Tahap ini sangat krusial karena kualitas fitur yang diekstraksi akan menentukan tingkat akurasi model pada proses pelatihan menggunakan Scikit-Learn. Dengan memanfaatkan landmark tangan sebagai representasi numerik dari bentuk gestur, sistem mampu mengubah citra visual menjadi data terstruktur yang siap digunakan dalam proses klasifikasi.

6.3 Pelatihan dan pengujian

```
1 import pickle
2 import numpy as np
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score
6
7
8 with open("data.pickle", "rb") as f:
9     data_dict = pickle.load(f)
10
11 data = np.array(data_dict['data'])
12 labels = np.array(data_dict['labels'])
13
14 print("Jumlah data:", len(data))
15
16 if len(data) == 0:
17     print("Dataset kosong! Jalankan ekstraksi fitur dulu.")
18     exit()
19
20
21 x_train, x_test, y_train, y_test = train_test_split(
22     *arrays: data,
23     labels,
24     test_size=0.1,
25     shuffle=True,
26     stratify=labels
```

```
28
29
30 model = RandomForestClassifier()
31 model.fit(x_train, y_train)
32
33
34 y_predict = model.predict(x_test)
35 score = accuracy_score(y_test, y_predict)
36
37 print(f'{score * 100:.2f}% of samples were classified correctly!')
38
39
40 with open("model.pickle", "wb") as f:
41     pickle.dump(model, f)
42
43 print("Model berhasil disimpan ke model.pickle")
```

Program yang dikembangkan merupakan tahap pelatihan (training) dan evaluasi model klasifikasi berbasis machine learning menggunakan algoritma Random Forest dari library Scikit-Learn. Tahap ini bertujuan untuk membangun model yang mampu mengenali pola dari

data fitur yang sebelumnya telah diekstraksi dari citra tangan dan disimpan dalam file data.pickle.

Pada bagian awal program dilakukan proses impor beberapa library penting, yaitu pickle untuk membaca data hasil ekstraksi fitur, numpy untuk mengelola data dalam bentuk array numerik, serta beberapa modul dari Scikit-Learn seperti RandomForestClassifier, train_test_split, dan accuracy_score. RandomForestClassifier digunakan sebagai algoritma klasifikasi karena memiliki kemampuan yang baik dalam menangani data dengan dimensi cukup besar serta mampu mengurangi risiko overfitting melalui mekanisme ensemble learning berbasis decision tree.

Dataset yang tersimpan dalam file data.pickle dibuka menggunakan fungsi open() dalam mode baca biner ("rb"), kemudian dimuat ke dalam variabel data_dict menggunakan pickle.load(). Data yang diperoleh terdiri dari dua komponen utama, yaitu fitur (data) dan label (labels). Fitur merepresentasikan koordinat landmark tangan hasil ekstraksi MediaPipe yang telah dinormalisasi, sedangkan label menunjukkan kelas atau kategori dari setiap data, misalnya huruf alfabet SIBI atau kondisi tertentu seperti buka dan tutup.

Selanjutnya, data dan label dikonversi menjadi array NumPy untuk mempermudah proses komputasi numerik. Program kemudian menampilkan jumlah data yang tersedia sebagai bentuk verifikasi awal. Apabila dataset kosong (jumlah data sama dengan nol), sistem akan menghentikan proses pelatihan dan menampilkan pesan bahwa ekstraksi fitur perlu dilakukan terlebih dahulu. Langkah ini penting untuk mencegah error selama proses training akibat tidak tersedianya data.

Tahap berikutnya adalah pembagian dataset menjadi data latih (training set) dan data uji (testing set) menggunakan fungsi train_test_split(). Parameter test_size=0.1 menunjukkan bahwa 10% dari total dataset digunakan sebagai data pengujian, sedangkan 90% sisanya digunakan untuk pelatihan model. Parameter shuffle=True memastikan data diacak sebelum dibagi agar distribusi lebih merata, sedangkan stratify=labels digunakan untuk menjaga proporsi setiap kelas tetap seimbang pada data latih dan data uji. Teknik stratifikasi ini sangat penting dalam kasus klasifikasi multikelas agar model tidak bias terhadap kelas tertentu.

Secara keseluruhan, program ini merupakan implementasi tahap supervised learning dalam pipeline sistem klasifikasi gesture. Proses pelatihan model menggunakan Random Forest memungkinkan sistem mempelajari pola hubungan antara fitur koordinat landmark tangan dan label kelasnya. Hasil evaluasi akurasi yang diperoleh nantinya dapat digunakan sebagai indikator performa model dalam mengenali gesture atau huruf alfabet secara otomatis. Tahap ini menjadi komponen krusial dalam pengembangan sistem pengenalan bahasa isyarat berbasis machine learning karena menentukan tingkat keakuratan dan keandalan sistem dalam implementasi real-time.

6.4 Implementasi

```
1  import pickle
2  import cv2
3  import mediapipe as mp
4  import numpy as np
5
6
7  with open("model.pickle", "rb") as f:
8      model = pickle.load(f)
9
10 cap = cv2.VideoCapture(0)
11
12 mp_hands = mp.solutions.hands
13 hands = mp_hands.Hands(static_image_mode=False, max_num_hands=1)
14
15 mp_drawing = mp.solutions.drawing_utils
16 mp_drawing_styles = mp.solutions.drawing_styles
17
18 while True:
19     data_aux = []
20     x_ = []
21     y_ = []
22
23     ret, frame = cap.read()
24     if not ret:
25         break
26
```

```
27     frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
28     results = hands.process(frame_rgb)
29
30     if results.multi_hand_landmarks:
31         for hand_landmarks in results.multi_hand_landmarks:
32
33             mp_drawing.draw_landmarks(
34                 frame,
35                 hand_landmarks,
36                 mp_hands.HAND_CONNECTIONS,
37                 mp_drawing_styles.get_default_hand_landmarks_style(),
38                 mp_drawing_styles.get_default_hand_connections_style()
39             )
40
41
42     fingers_open = 0
43
44     if hand_landmarks.landmark[8].y < hand_landmarks.landmark[6].y:
45         fingers_open += 1
46     if hand_landmarks.landmark[12].y < hand_landmarks.landmark[10].y:
47         fingers_open += 1
48     if hand_landmarks.landmark[16].y < hand_landmarks.landmark[14].y:
49         fingers_open += 1
50     if hand_landmarks.landmark[20].y < hand_landmarks.landmark[18].y:
51         fingers_open += 1
52
```

```
51         fingers_open += 1
52
53     if fingers_open >= 3:
54         label = "BUKA"
55         color = (0, 255, 0)
56     else:
57         label = "TUTUP"
58         color = (0, 0, 255)
59
60
61     cv2.putText(
62         frame,
63         label,
64         org: (50, 100),
65         cv2.FONT_HERSHEY_SIMPLEX,
66         fontScale: 1.5,
67         color,
68         thickness: 3
69     )
70
71     cv2.imshow( winname: "Deteksi Telapak Tangan", frame)
72
73     if cv2.waitKey(1) & 0xFF == ord('q'):
74         break
75
76 cap.release()
```


7. Referensi

- Bazarevsky, V., et al. (2020). *BlazePose: On-device Real-time Body Pose tracking*. Google AI.
- Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (t.t.). *MediaPipe: A Framework for Building Perception Pipelines*.
- Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). *MediaPipe Hands: On-device Real-time Hand Tracking* (arXiv:2006.10214). arXiv. <https://doi.org/10.48550/arXiv.2006.10214>
- OpenCV Documentation. (2023). *Open Source Computer Vision Library*